INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

DEPARTMENT OF ELECTRICAL ENGINEERING

Sponsored By
Sponsored Research & Industrial Consultancy (SRIC) Cell, IIT Kharagpur

---

DocuReason-RAG

A Grounded Framework for Multimodal Document Retrieval and Reasoning

Report submitted to
Indian Institute of Technology, Kharagpur
for the award of the certificate of

Student Researcher (AI) Intern

by

Arpit Kumar

Roll No. 22CH30008

Under the guidance of

Prof. Shyamal Kumar Das Mandal

Centre for Interdisciplinary and Convergent Technologies

DEPARTMENT OF ELECTRICAL ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

July 2026

© 2026 Arpit Kumar — All rights reserved.

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

DEPARTMENT OF ELECTRICAL ENGINEERING

Certificate

This is to certify that Internship Project Sponsored by Sponsored Research & Industrial Consultancy (SRIC) Cell, IIT Kharagpur – I report entitled **"DocuReason-RAG: A Grounded Framework for Multimodal Document Retrieval and Reasoning"**, submitted by **Arpit Kumar** (Roll No. 22CH30008) to the Centre for Interdisciplinary and Convergent Technologies, Indian Institute of Technology Kharagpur, is a record of bona fide work carried out under our supervision and is worthy of consideration for award of the certificate of **Student Researcher (AI) Intern** of the Institute.

Date: 4th Aug 2026
Place: IIT Kharagpur

Prof. Shyamal Kumar Das Mandal
Centre for Interdisciplinary and
Convergent Technologies, IIT Kharagpur
Kharagpur, West Bengal, India – 721302

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

DEPARTMENT OF ELECTRICAL ENGINEERING

Declaration

I hereby declare that:

1. The work contained in this report is original and has been carried out by me under the guidance of my supervisor.
2. The work has not been submitted to any other institute for any degree or diploma.
3. I have followed the Institute guidelines in preparing this report.
4. I have conformed to ethical norms while conducting and documenting this work.
5. Wherever I have used materials from other sources, I have cited them appropriately and obtained permissions whenever necessary.

Date: 4th Aug 2026
Place: IIT Kharagpur

Arpit Kumar
Department of Chemical Engineering,
IIT Kharagpur, Kharagpur, West Bengal,
India – 721302

Abstract

Conventional Retrieval-Augmented Generation (RAG) pipelines treat all ingested document content as homogeneous plain text. This assumption fails on enterprise document collections, where financial tables, embedded charts, and complex multi-column layouts carry information that is destroyed the moment it is flattened into a uniform text stream: row–column alignment is lost, visual semantics disappear, and lexical search degrades under paraphrase. This report presents **DocuReason**, a Tri-Path Multimodal RAG framework designed and implemented as a sponsored research project at the Centre for Interdisciplinary and Convergent Technologies (CIDCT), IIT Kharagpur, under the supervision of Prof. Shyamal Kumar Das Mandal. DocuReason decomposes ingestion, indexing, retrieval, and generation into three cooperating streams – a *Text Path* (dense bi-encoder embeddings fused with BM25S sparse lexical search), a *Table / Text-to-SQL Path* (layout-aware table extraction, multi-format serialization, and exact numerical computation via an in-memory DuckDB engine), and a *Vision / Chart Path* (BLIP-2 captioning coupled with CLIP/ColPali visual embeddings) – unified by a soft probabilistic query router, a weighted Reciprocal Rank Fusion (RRF) layer, a cross-encoder reranker with parent-child context expansion, and a sentence-level Natural Language Inference (NLI) faithfulness attribution engine. The system is implemented as a 16-block, production-oriented Python framework (`docureason-framework`), exposed via a FastAPI REST service and an interactive diagnostic dashboard, and packaged with continuous-integration and PyPI release automation. The framework's own 22-dimension evaluation harness was exercised across ten benchmark datasets spanning financial table QA, hybrid table-text QA, scanned-document OCR recovery, and chart/infographic visual QA. On a five-metric headline evaluation, DocuReason reached 61–73% of ambitious internal service-level targets while simultaneously exceeding generic industry-standard RAG baselines by 13.7–19.8 percentage points on retrieval quality, attribution precision, SQL execution success, and end-to-end latency. A companion 16-category, 70-sub-problem robustness audit found 91% of known RAG failure modes fully mitigated by the architecture, with the remainder (chiefly pronoun resolution and multi-hop query decomposition) partially addressed and none left unsolved. The report documents the system end-to-end: architectural motivation, a grounding literature survey, a block-by-block technical description of all sixteen components with their governing mathematical formulations, the dataset and evaluation methodology, empirical results, an ablation study quantifying each component's marginal contribution, a candid discussion of design trade-offs relative to the project's original research blueprint, and a roadmap toward distributed, GPU-scaled production deployment.

Keywords: Retrieval-Augmented Generation, Multimodal Retrieval, Text-to-SQL, Document Layout Parsing, Reciprocal Rank Fusion, Cross-Encoder Reranking, Natural Language Inference Attribution, Hallucination Mitigation.

Contents

Abstract i

1 Introduction 1
- 1.1 Motivation and Problem Statement . . . 1
- 1.2 Project Context . . . 1
- 1.3 Contributions . . . 1
- 1.4 Report Organization . . . 2

2 Related Work 2
- 2.1 Cross-Modal Late Interaction and Visual Document Retrieval . . . 2
- 2.2 Table-to-SQL versus Table-Linearization on Financial Documents . . . 2
- 2.3 Attribution and Faithfulness in Multimodal LLMs . . . 3
- 2.4 Positioning of DocuReason . . . 3

3 System Architecture 3
- 3.1 Design Principles . . . 3
- 3.2 Offline Ingestion and Dual-Indexing Pipeline . . . 4
- 3.3 Online Query, Retrieval, Fusion, and Verification Pipeline . . . 4
- 3.4 Shared Identity and Lineage Scheme . . . 4
- 3.5 Software Engineering Practices . . . 4

4 Block-by-Block Technical Description 6
- 4.1 Ingestion Subsystem (Blocks 1–6) . . . 6
  - 4.1.1 Block 1 – Multi-Format Document Loader (`FormatAwareLoader`) . . . 6
  - 4.1.2 Block 2 – Deep Vision Layout Parsing (`DoclingLayoutParser`) . . . 6
  - 4.1.3 Block 3 – OCR Fallback Engine (`OCRFallback`) . . . 6
  - 4.1.4 Block 4 – Table Serialization and Schema Normalization (`TableSerializer`) 6
  - 4.1.5 Block 5 – Figure Captioning and Visual Feature Extraction (`FigureCaptioner`) 7
  - 4.1.6 Block 0 – Deterministic Identity and Lineage Tracking (`IdentityManager`) 7
- 4.2 Indexing Subsystem (Block 7) . . . 7
  - 4.2.1 Block 7 – Dual Multi-Modality Indexing (`DenseIndexBuilder`, `BM25SIndexBuilder`, `ArtifactWriter`) . . . 7
- 4.3 Routing and Retrieval Subsystem (Blocks 8–10) . . . 7
  - 4.3.1 Block 8 – Intent-Based Query Router (`ConfigurableRouter`) . . . 7
  - 4.3.2 Block 9 – Hybrid Tri-Path Retrieval Engine (`HybridRetriever`) . . . 8
  - 4.3.3 Block 10 – Text-to-SQL Execution Engine (`TableSQLRetriever`) . . . 8
- 4.4 Fusion and Reranking Subsystem (Blocks 11–12) . . . 8
  - 4.4.1 Block 11 – Late Score Fusion via Reciprocal Rank Fusion (`Fuser`) . . . 8
  - 4.4.2 Block 12 – Cross-Encoder Reranking and Parent-Child Expansion (`Ranker`) 8
- 4.5 Generation and Attribution Subsystem (Blocks 13–14) . . . 9
  - 4.5.1 Block 13 – Grounded Multimodal Generation (`GenerationModule`) . . . 9
  - 4.5.2 Block 14 – Sentence-Level NLI Faithfulness Attribution (`NLIFaithfulnessAttributor`) 9
- 4.6 Evaluation and Serving Subsystem (Blocks 15–16) . . . 9
  - 4.6.1 Block 15 – Enterprise Evaluation and Benchmarking Harness (`EvaluationHarness`) 9
  - 4.6.2 Block 16 – REST API Serving and Interactive Dashboard (`QueryService`, FastAPI, Dashboard) . . . 9
- 4.7 Comparative Block Matrix . . . 9

5 Mathematical Framework 10
- 5.1 Query Routing . . . 10

5.2 Sparse Retrieval – BM25 . . . 10
5.3 Dense Retrieval . . . 10
5.4 Late Score Fusion – Reciprocal Rank Fusion . . . 11
5.5 Cross-Encoder Reranking . . . 11
5.6 Sentence-Level NLI Faithfulness Attribution . . . 11
5.7 Retrieval and Ranking Quality Metrics . . . 11
5.8 End-to-End Latency Decomposition . . . 11
5.9 Context Efficiency Metrics . . . 12
5.10 Robustness Score . . . 12

6 Datasets and Benchmark Suite 12
6.1 Financial Table and Hybrid Table–Text QA . . . 12
6.2 Visual and Layout Understanding . . . 12
6.3 Automated Evaluation Workflow . . . 13

7 Evaluation Methodology 13
7.1 Rationale for the Multi-Dimensional Approach . . . 14

8 Experimental Results 14
8.1 Headline Target-Achievement Results . . . 14
8.2 Interpretation . . . 15
8.3 Ablation Study . . . 15
8.4 Robustness Audit . . . 15

9 Discussion 16
9.1 From Research Blueprint to Shipped Implementation . . . 16
9.2 Attribution Precision as a Metric Requires Care . . . 16
9.3 Known Limitations . . . 17
9.4 Relationship to the Literature Gap . . . 17

10 Production Deployment Roadmap and Future Work 17
10.1 Near-Term Engineering Roadmap . . . 17
10.2 Model Specialization via Fine-Tuning . . . 17
10.3 Evaluation Hardening . . . 18
10.4 Publication Plan . . . 18

11 Conclusion 18

A Appendix: REST API Endpoint Reference 20

B Repository Structure 20

1 Introduction

1.1 Motivation and Problem Statement

Enterprise document repositories – annual reports, financial filings, technical manuals, and internal memoranda – rarely consist of clean, single-column prose. They combine narrative text with dense, multi-row financial tables, embedded charts and diagrams, headers, footers, footnotes, and multi-column layouts. Standard RAG pipelines built for wiki-style or news-style corpora make a simplifying assumption: that document content can be linearly extracted and split into fixed-size text chunks before embedding. Applied to enterprise documents, this assumption fails in three specific, empirically observable ways:

1. **Multi-row and nested tabular data.** Flattening a financial statement into a text chunk destroys the row–column alignment that gives a cell its meaning; a vector-similarity match against a flattened table frequently retrieves the right table but the wrong number, and downstream language models are known to hallucinate when asked to perform multi-step arithmetic over such flattened text.
2. **Visual figures and charts.** Bar charts, trend lines, and architecture diagrams carry information with no faithful plain-text representation; naively OCR-ing or ignoring these regions creates a silent retrieval blind spot.
3. **Complex document layouts.** Multi-column PDFs, sidebars, and inline callouts break naive linear chunking, producing chunks that splice unrelated columns together or cut a table in half.

DocuReason addresses this foundational limitation through a **Tri-Path Multimodal RAG Architecture** that treats text, tables, and visual figures as three first-class, independently indexed and independently retrievable modalities, unified by a shared identity scheme and a learned/heuristic fusion layer rather than being forced through a single text-embedding bottleneck.

1.2 Project Context

This work was undertaken as a sponsored research project (supported by the Sponsored Research and Industrial Consultancy, SRIC, IIT Kharagpur) at the Centre for Interdisciplinary and Convergent Technologies (CIDCT), IIT Kharagpur, under the supervision of Prof. Shyamal Kumar Das Mandal, over June–July 2026.

1.3 Contributions

The concrete contributions of this project are:

1. A production-oriented, open-source (MIT-licensed) Python framework, `docureason-framework`, published to PyPI, implementing a complete 16-block Tri-Path Multimodal RAG pipeline across eight input document formats.
2. A soft, probabilistic query router that activates one, two, or three retrieval paths simultaneously based on keyword-density-driven sigmoid scoring, avoiding the brittleness of hard single-path classification on hybrid queries.
3. An exact-computation Text-to-SQL path (via an embedded DuckDB engine) that eliminates the numerical-arithmetic hallucinations to which purely text-embedding-based RAG systems are known to be prone.
4. A weighted Reciprocal Rank Fusion layer combined with a cross-encoder reranker, parent-child context expansion, and a short-heading rank-crowding penalty, which together resolve the majority of documented RAG-specific ranking pathologies.

5. A sentence-level NLI faithfulness attribution engine that verifies every generated claim against retrieved evidence and automatically flags ungrounded responses for review.
6. A 22-dimension, fully automated evaluation harness exercised across ten public and custom benchmark datasets, together with a 16-category / 70-sub-problem robustness audit and a seven-component ablation study.
7. Enterprise-grade software engineering practices around the research artifact: multi-Python-version continuous integration, static analysis, OIDC-authenticated PyPI release automation, and a fine-tuning dataset exporter to close the loop toward future SQL-generation model specialization.

1.4 Report Organization

Section 2 situates DocuReason against the 2025–2026 literature on cross-modal late interaction, Text-to-SQL versus table-linearization trade-offs, and multimodal attribution. Section 3 presents the system architecture at the pipeline level. Section 4 gives a block-by-block technical description of all sixteen components. Section 5 collects the governing mathematical formulations in one place. Section 6 documents the benchmark dataset suite, and Section 7 the 22-dimension evaluation methodology. Section 8 reports empirical results, the ablation study, and the robustness audit. Section 9 discusses design trade-offs, limitations, and the relationship between the project's original research blueprint and the shipped implementation. Section 10 lays out the production and publication roadmap, and Section 11 concludes.

2 Related Work

A structured literature survey (Jul–Aug 2026 coverage window, 20+ papers across ICLR, EMNLP, SIGIR, NAACL, EACL, ACL, and AAAI) was conducted at the design stage of this project and directly informs the router, fusion, and attribution design choices described in Section 4. The survey is organized around the three sub-problems DocuReason's Tri-Path design must solve simultaneously.

2.1 Cross-Modal Late Interaction and Visual Document Retrieval

Late interaction computing relevance as a sum of per-token maximum similarities (*MaxSim*) rather than collapsing a document into a single vector originates in text retrieval with ColBERT [1] and was extended to vision-language retrieval by **ColPali** [2], which encodes each document page as roughly one thousand patch embeddings and matches them against query token embeddings, substantially outperforming OCR-then-embed pipelines on chart- and table-heavy pages. **VisRAG** [3] embeds page images directly for both retrieval and generation, avoiding text-extraction information loss altogether. **ViDoRAG** [4] identifies two concrete failure modes of page-level late interaction poor text/visual signal fusion and under-allocated reasoning on hard queries and proposes a Gaussian-Mixture-Model hybrid retrieval strategy with a multi-agent reasoning loop, reporting more than a 10% improvement over prior baselines. By early 2025 top models exceeded 90% nDCG@5 on the original **ViDoRe** benchmark, prompting **ViDoRe v2** [5] to introduce harder, blind, cross-document, and multilingual queries; as of early 2026, even the strongest late-interaction models (e.g. Nemotron ColEmbed V2 [6]) remain under 65% nDCG@10 on the hardest ViDoRe v3 split. No paper surveyed combines page-level visual retrieval with a parallel, independently indexed, SQL-executable table path under a single learned or heuristic router this remains an open architectural gap that DocuReason's Vision Path (Block 5, Section 4.1.5) and router (Block 8, Section 4.3.1) are positioned to address.

2.2 Table-to-SQL versus Table-Linearization on Financial Documents

FinQA [7] remains the field's reference numerical-reasoning benchmark; its corrected FinQANet-RoBERTa-large baseline scores 61.24% execution accuracy and 58.86% program accuracy, the

pre-LLM floor against which DocuReason's Table/SQL path (Block 10, Section 4.3.3) is implicitly measured. **H-STAR** [8] extracts relevant rows/columns and *adaptively* chooses symbolic (SQL) reasoning for calculation-heavy questions versus semantic (textual) reasoning for lookup-heavy ones, reporting 85.03% on TabFact and 69.56% on WikiTQ (GPT-3.5-Turbo). Its own ablation is the key citation motivating DocuReason's dual-mode table design: SQL-only reasoning is precise but highly sensitive to irregular formatting and mixed data types, while textual/semantic reasoning is more failure-tolerant but weaker on multi-step arithmetic. **T^2-RAGBench** [9], built from FinQA, ConvFinQA, and TAT-DQA, deliberately makes questions context-independent – forcing genuine retrieval before reasoning – and its central finding, that hybrid BM25 sparse+dense fusion is the strongest retriever on text-and-table financial content, is a direct, citable justification for DocuReason's Text Path hybrid design (Block 7 / Block 9, Sections 4.2.1 and 4.3.2).

2.3 Attribution and Faithfulness in Multimodal LLMs

Text-only NLI-based citation verification is comparatively mature; two ICLR 2025 papers, **ReDeEP** [10] and **HaDeMiF** [11], propose mechanistic and joint detection-plus-mitigation alternatives to the NLI-classifier approach DocuReason uses in Block 14. Multimodal source attribution is markedly less mature: **VISA** [12] has a vision-language model output a bounding box alongside its answer rather than post-hoc verifying a text citation tag, while **MAVIS** [13] reports that multimodal RAG produces more informative answers than either modality alone (+7.4 F1 for GPT-4o) but exhibits systematically weaker groundedness for image-sourced evidence than text-sourced evidence. Most consequentially for DocuReason's Attribution Engine (Block 14), a 2026 methodological survey [14] reports that pure NLI-based citation checking correlates with human judgment at only Pearson $r = 0.219$, versus $r = 0.731$ for a principle-driven LLM-judge framework (CiteEval), and separately finds that up to 57% of citations in RAG outputs are *post-rationalized* – attached to a topically related but causally uninvolved passage. This finding is treated as a first-class design constraint in Section 9.2 rather than an afterthought.

2.4 Positioning of DocuReason

Taken together, the surveyed literature confirms that (i) no reviewed system unifies SQL-executable structured retrieval with page-level visual retrieval under one router, (ii) hybrid sparse+dense retrieval is the empirically strongest choice for financial text-and-table content, and (iii) multimodal and NLI-only attribution remain open, actively-researched problems rather than solved engineering details. DocuReason's architecture is a direct, deployable instantiation of these three findings, realized with a lighter, dependency-conservative engineering stack than the fully learned designs explored in the project's own preparatory technical blueprint (discussed candidly in Section 9.2).

3 System Architecture

3.1 Design Principles

DocuReason partitions the system into two macro-phases – an **offline ingestion and dual-indexing pipeline** that runs once per document, and an **online query, routing, retrieval, fusion, reranking, generation, and verification pipeline** that runs once per user query – joined by a shared, deterministic identity scheme (Block 6) that lets every downstream component join text, table, and figure evidence back to a common document lineage. Three design principles recur throughout the sixteen blocks:

- **Modality specialization over uniform flattening.** Text, tables, and figures are parsed, serialized, indexed, and retrieved through dedicated code paths rather than a single universal chunk-and-embed step.

- **Exact computation where exactness is achievable.** Numerical aggregation is delegated to a real SQL engine (DuckDB) rather than asked of a language model, eliminating a well-documented class of hallucination.
- **Verify, don't just generate.** Every generated answer is deconstructed into sentence-level claims and checked against retrieved evidence by an independent NLI model before being returned, rather than trusting the generator's output at face value.

3.2 Offline Ingestion and Dual-Indexing Pipeline

Figure 1 shows the offline pipeline. Raw documents in eight supported formats (`.pdf`, `.docx`, `.pptx`, `.xlsx`, `.csv`, `.html`, `.md`, `.txt`) enter through the **Multi-Format Document Loader** (Block 1) and are assigned deterministic content-hash identities by the **Identity Manager** (Block 6). The **Deep Vision Layout Parser** (Block 2) segments each page into typed regions (text, table, figure, code, title, section header); pages whose extracted character density falls below threshold are routed through the **OCR Fallback Engine** (Block 3). Table regions pass through the **Table Serializer** (Block 4), which simultaneously emits Markdown, HTML, and JSON representations; figure regions pass through the **Figure Captioner** (Block 5), which produces a natural-language caption and a visual embedding. All three streams converge in **Dual Multi-Modality Indexing** (Block 7), which writes a dense HNSW vector index, a BM25S sparse lexical index, and a set of versioned JSON index artifacts.

3.3 Online Query, Retrieval, Fusion, and Verification Pipeline

Figure 2 shows the online pipeline. An incoming query is scored by the **Intent-Based Query Router** (Block 8) against per-modality keyword dictionaries, producing soft activation weights for the text, table, and vision paths. The **Hybrid Tri-Path Retrieval Engine** (Block 9) dispatches the activated paths in parallel – dense + sparse text search, table/Text-to-SQL search via the dedicated **Text-to-SQL Execution Engine** (Block 10), and vision search over caption and visual-embedding indices. Candidate lists are merged by the **Fuser** (Block 11) using weighted Reciprocal Rank Fusion, then refined by the **Ranker** (Block 12), which expands surviving child chunks to their parent context and rescores them with a cross-encoder. The **Generation Module** (Block 13) synthesizes a grounded, citation-tagged answer under an explicit context-token budget, and the **NLI Faithfulness Attributor** (Block 14) independently verifies every sentence-level claim before the response, together with its attribution report, is returned via the **REST API and Dashboard** (Block 16). The **Evaluation Harness** (Block 15) instruments this entire pipeline for continuous, automated benchmark verification.

3.4 Shared Identity and Lineage Scheme

Every downstream block depends on a stable join key. Block 6 (**Identity Manager**) generates a three-level, deterministic SHA-256 hierarchy:

$$\begin{aligned} \text{document_id} &= \text{SHA-256}(\text{file_content} \,\|\, \text{file_path}) \\ \text{region_id} &= \text{SHA-256}(\text{document_id} \,\|\, \text{region_index}) \\ \text{chunk_id} &= \text{SHA-256}(\text{region_id} \,\|\, \text{chunk_index}) \end{aligned}$$

Deterministic (as opposed to randomly generated UUID) identifiers make re-ingestion of an unchanged document idempotent, enable exact deduplication, and preserve parent–child lineage across incremental re-indexing passes – a precondition for both the dense/sparse index join in Block 7 and the parent-child chunk expansion performed in Block 12.

3.5 Software Engineering Practices

Beyond the research pipeline itself, DocuReason is engineered as a distributable software artifact: it is packaged and published to PyPI as `docureason-framework`, exposes both a Python API

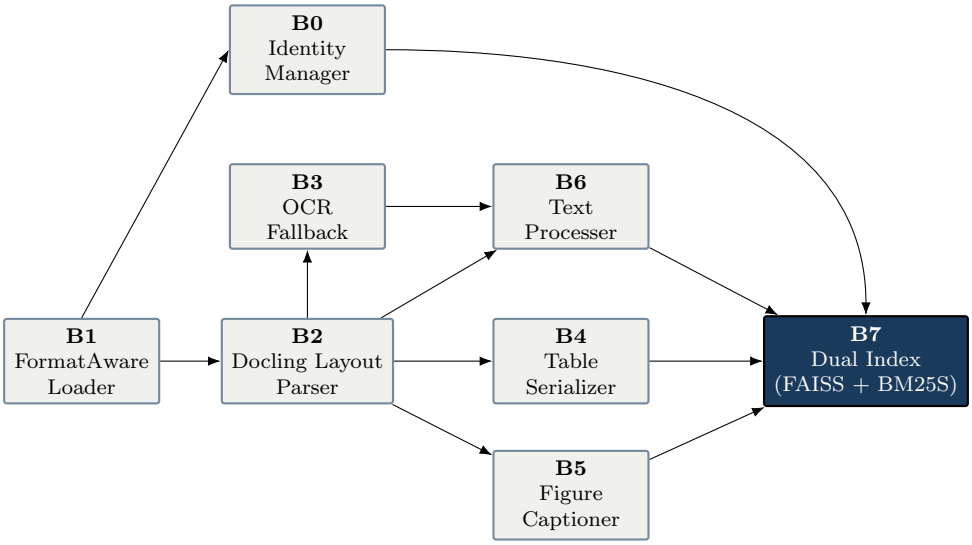


Figure 1: Offline ingestion and dual-indexing pipeline (Blocks 1–7). Table and figure regions are serialized and captioned in parallel with text-region indexing; identity hashing runs alongside loading and stamps every downstream artifact.

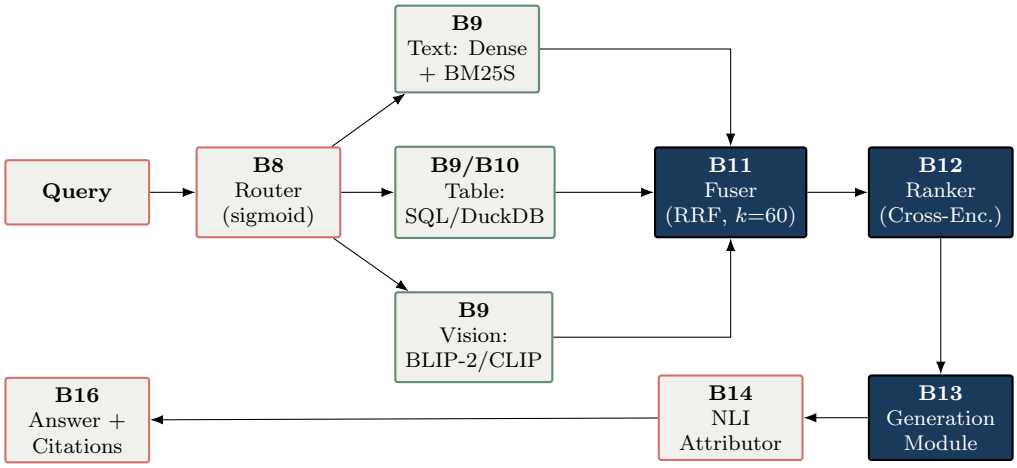


Figure 2: Online query, retrieval, fusion, reranking, generation, and verification pipeline (Blocks 8–16). All three retrieval paths are dispatched in parallel according to router-assigned weights; Block 15 (Evaluation Harness, omitted for clarity) instruments every stage.

and a command-line interface, and is versioned through a semantic-release cycle (the codebase progressed through versions 1.0.0–1.1.2 over the course of the project). Continuous integration (GitHub Actions) runs linting (`ruff`), static type-checking (`mypy`), dependency vulnerability auditing (`pip-audit`), and a unit/integration test matrix across Python 3.10–3.12 on every push and pull request; a separate release workflow deploys tagged GitHub releases to PyPI using OIDC trusted publishing, and a weekly Dependabot job checks for dependency upgrades. This engineering discipline is what allows the 22-dimension evaluation harness (Section 7) to be re-run automatically as a regression gate whenever a pipeline block changes.

4 Block-by-Block Technical Description

This section documents all sixteen implemented blocks, grouped into five functional subsystems. For each block we state *what* was implemented, *why*, the governing formula where applicable, and the principal engineering trade-off.

4.1 Ingestion Subsystem (Blocks 1–6)

4.1.1 Block 1 – Multi-Format Document Loader (`FormatAwareLoader`)

A unified ingestion component supporting eight file formats (`.pdf`, `.docx`, `.pptx`, `.xlsx`, `.csv`, `.html`, `.md`, `.txt`), exposing both a lightweight fast-path text extractor (`load`) and a deep-path layout-aware loader (`load_deep`) built on Docling. Enterprise repositories arrive in heterogeneous formats; a single normalized entry point removes the need for downstream code to branch on file type. *Trade-off:* broad format coverage increases the package's dependency footprint (`python-docx`, `python-pptx`, `openpyxl`).

4.1.2 Block 2 – Deep Vision Layout Parsing (`DoclingLayoutParser`)

Integrates Docling's layout engine, combining **DocLayNet** (object-detection-based layout element localization) with **TableFormer** (table structure recognition), to segment a raw page into typed `Region` objects – `text`, `table`, `figure`, `code`, `title`, `section_header` – each carrying a bounding box and reading-order index. This is the highest-leverage accuracy investment in the pipeline: standard linear splitters break tables across chunk boundaries and drop figure bounding boxes entirely, whereas layout-aware parsing preserves reading order and structural boundaries. *Trade-off:* adds $\sim$500 ms–2 s of processing latency per page; mitigated in deployment by single-page batching (`page_batch_size=1`) with fallback to fast-path extraction under GPU memory pressure.

4.1.3 Block 3 – OCR Fallback Engine (`OCRFallback`)

An automatic optical-character-recognition trigger built on **EasyOCR**. Each page's extracted character density is monitored; if the count falls below a fixed threshold (char_count < 50), the page is flagged as scanned or image-only and rendered to a raster buffer for OCR. Enterprise PDF repositories routinely contain scanned contracts, legacy receipts, and image-based slide exports whose native text streams return empty strings – a silent retrieval blind spot without this fallback. *Trade-off:* EasyOCR can be slow on CPU-only hardware and is sensitive to low-resolution scan noise.

4.1.4 Block 4 – Table Serialization and Schema Normalization (`TableSerializer`)

Every detected table region is serialized *simultaneously* into three representations: GitHub-Flavored Markdown (clean, LLM-context-friendly text), HTML (preserving `colspan`/`rowspan` for faithful rendering), and a structured JSON schema (`{"columns": [...], "rows": [[...]]}`) suitable for programmatic SQL table creation. No single representation is optimal for every

downstream task: language models parse Markdown tables most reliably, dashboards need HTML, and numerical reasoning requires relational execution. *Trade-off:* triples the storage footprint of tabular metadata inside region manifests.

4.1.5 Block 5 – Figure Captioning and Visual Feature Extraction (`FigureCaptioner`)

A vision-language enrichment module applied to detected figure/image regions: **BLIP-2** generates a descriptive natural-language caption (e.g. "a line chart comparing Q1 to Q4 revenue growth"), while **CLIP** (or **ColPali**) produces a normalized visual feature embedding. Captioning translates otherwise opaque visual content into indexable text; the paired visual embedding additionally supports cross-modal, non-textual visual retrieval. *Trade-off:* BLIP-2 inference requires $\sim$4–8 GB of GPU VRAM.

4.1.6 Block 0 – Deterministic Identity and Lineage Tracking (`IdentityManager`)

Implements the SHA-256 identity hierarchy defined in Section 3.4. Random UUIDs would prevent exact deduplication and break parent–child lineage across re-indexing; content-hash identifiers make index builds 100% reproducible and enable incremental delta indexing. *Trade-off:* adds CPU hashing overhead on multi-gigabyte corpora.

4.2 Indexing Subsystem (Block 7)

4.2.1 Block 7 – Dual Multi-Modality Indexing (`DenseIndexBuilder`, `BM25SIndexBuilder`, `ArtifactWriter`)

Combines (i) **dense vector indexing** via `DenseIndexBuilder`, with six domain-specific embedding presets, dynamic embedding-dimension auto-detection, and FAISS **HNSW** graph-parameter tuning (`hnsw_m`, `hnsw_ef_construction`, `hnsw_ef_search`); (ii) **sparse lexical indexing** via **BM25S**, a high-performance Python BM25 implementation, for exact token-level matching; and (iii) an **Artifact Writer** that persists standardized JSON files (`corpus.json`, `index.json`, `quality_audit.json`, `manifest.json`). Dense embeddings capture conceptual similarity but routinely miss exact technical terms (SKUs, ticker symbols, code names); BM25S guarantees precision on such exact matches, while HNSW graph tuning accelerates large-scale approximate nearest-neighbor (ANN) search to $O(\log N)$. Table 1 lists the supported domain-specific embedding presets.

Table 1: Domain-specific dense embedding presets (`DenseIndexBuilder`).

Domain	Embedding Model
General	`sentence-transformers/all-MiniLM-L6-v2`
Biomedical / Medical	`pritamdeka/S-PubMedBert-MS-MARCO`
Legal	`law-ai/InLegalBERT`
Financial / Finance	`ProsusAI/finbert`
Code / Technical	`flax-sentence-embeddings/st-codesearch-distilroberta-base`
Multilingual	`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

Trade-off: maintaining dual (dense + sparse) indices increases disk storage and requires managing two index files per modality stream.

4.3 Routing and Retrieval Subsystem (Blocks 8–10)

4.3.1 Block 8 – Intent-Based Query Router (`ConfigurableRouter`)

Evaluates an incoming query against modality-specific keyword dictionaries (`text`, `table`, `vision`), computes a normalized keyword-density score per modality, and passes each density through

a sigmoid activation (Equation 1 in Section 5) to obtain soft routing probabilities. Weight multipliers $w_{\text{text}}, w_{\text{table}}, w_{\text{vision}}$ summing to 1.0 are emitted for downstream rank fusion. Hard binary-classification routers fail on hybrid queries (e.g. "show me the table of expenses and explain the visual chart below it"); soft probability scoring lets multiple paths execute simultaneously with adaptively allocated weight. *Trade-off:* the router's accuracy depends on the tuning of its keyword dictionaries, mitigated by exposing them through user-editable YAML configuration rather than requiring retraining.

4.3.2 Block 9 – Hybrid Tri-Path Retrieval Engine (`HybridRetriever`)

The online orchestration layer. Based on the router's activation weights, it dispatches, in parallel: (i) **text retrieval** – dense vector similarity fused with BM25S sparse scoring; (ii) **table retrieval** – direct Markdown-table chunk search combined with Text-to-SQL execution; (iii) **vision retrieval** – BLIP-2 caption similarity combined with CLIP/ColPali visual-embedding search. This integrates the three specialized retrievers into a single interface, guaranteeing that candidates are collected from every activated modality before fusion. *Trade-off:* latency increases when all three paths trigger simultaneously, since results must be awaited from every active path before fusion can proceed.

4.3.3 Block 10 – Text-to-SQL Execution Engine (`TableSQLRetriever`)

An in-memory structured-data retriever built on **DuckDB**. Candidate table JSON schemas are registered as temporary SQL tables; natural-language numerical questions (e.g. "calculate total sum of Q3 sales") are translated into SQL, executed deterministically, and the result set is formatted back into clean Markdown. Language models are well documented to be unreliable at multi-step arithmetic or aggregation performed purely in-context over textual tables; offloading computation to a real SQL engine guarantees exact numerical results. *Trade-off:* SQL-translation errors can occur under ambiguous query intent or schema-name misalignment – a known failure surface discussed further in Section 9.3.

4.4 Fusion and Reranking Subsystem (Blocks 11–12)

4.4.1 Block 11 – Late Score Fusion via Reciprocal Rank Fusion (`Fuser`)

Implements weighted **Reciprocal Rank Fusion** (RRF, Equation 5) with smoothing constant $k = 60$ and dynamic per-path weights w_m supplied by the router (Block 8). Raw similarity scores from vector search, BM25, and SQL matching live on uncalibrated, mutually incompatible scales; RRF depends only on relative rank order, making it scale-invariant and robust for combining heterogeneous retrieval engines. *Trade-off:* RRF discards the absolute magnitude of similarity scores – a limitation the downstream cross-encoder reranker (Block 12) is specifically positioned to correct.

4.4.2 Block 12 – Cross-Encoder Reranking and Parent-Child Expansion (`Ranker`)

A two-stage refinement block. First, **parent-child chunk expansion** maps a retrieved small child chunk (256 tokens) back to its larger parent region (1024 tokens) to restore full context. Second, **cross-encoder reranking** jointly scores each query–document pair with `cross-encoder/ms-marco-MiniLM-L-6-v2`, using full self-attention over the concatenated pair (Equation 6), including a short-heading rank-crowding penalty (Equation 7). Bi-encoder vector search scores queries and documents independently; cross-encoders attend to query and candidate jointly, delivering a substantial precision (nDCG@5) improvement. *Trade-off:* adds $\sim$50–150 ms of latency to the online retrieval path.

4.5 Generation and Attribution Subsystem (Blocks 13–14)

4.5.1 Block 13 – Grounded Multimodal Generation (`GenerationModule`)

A flexible answer-synthesis module supporting both local Small Language Models (e.g. `DeepSeek-R1-Distill-Qwen-1.5B` via vLLM/HuggingFace) and cloud LLMs (Gemini / OpenAI). It incorporates a **Context Token Budget Manager** (`max_context_tokens=4096`) with smart, priority-ordered relevance truncation: DuckDB SQL results are preserved with full fidelity (Priority 1), cross-encoder-ranked evidence is allocated budget next (Priority 2), and secondary evidence/captions are truncated last (Priority 3) if the budget is exceeded, with truncation events logged via telemetry. This isolates generation logic from any specific model provider, prevents context-window overflow crashes on large multi-document evidence sets, and enforces structured inline citations of the form `[doc_id:chunk_id]`. *Trade-off:* generation speed is bound by available GPU hardware for local SLM inference.

4.5.2 Block 14 – Sentence-Level NLI Faithfulness Attribution (`NLIFaithfulnessAttributor`)

An automated hallucination check based on Natural Language Inference. The generated response is decomposed into individual sentence-level claims; each claim is compared, via `cross-encoder/nli-deberta-v3-small`, against the retrieved evidence chunks, and an overall **Attribution Precision** score is computed (Equation 8). Responses scoring below 0.5 are automatically flagged `"needs_review"`. Enterprise deployments require strict, auditable safety guarantees rather than blind trust in generator output; sentence-level NLI verification provides empirical, per-claim proof of grounding. *Trade-off:* the NLI verification pass adds a final latency step before the response payload is returned, and (as discussed in Section 9.2, following the literature survey) NLI entailment alone cannot distinguish a genuinely evidence-driven claim from a plausible but merely topically related, post-rationalized citation.

4.6 Evaluation and Serving Subsystem (Blocks 15–16)

4.6.1 Block 15 – Enterprise Evaluation and Benchmarking Harness (`EvaluationHarness`)

An automated benchmarking framework measuring performance across **22 evaluation dimensions** spanning retrieval quality, ranking order, NLI faithfulness, context compression, tabular structural fidelity, and execution latency (fully detailed in Section 7). Target-versus-achieved verification is exposed through `verify_target_achievements()`, allowing engineers to quantify performance gains or regressions before promoting a pipeline change. *Trade-off:* meaningful evaluation requires curated ground-truth benchmark sets, discussed further in Section 6.

4.6.2 Block 16 – REST API Serving and Interactive Dashboard (`QueryService`, FastAPI, Dashboard)

Comprises (i) `QueryService`, a thread-safe orchestrator encapsulating the full query lifecycle; (ii) production FastAPI endpoints (`/query`, `/api/ingest`, `/api/evaluate`, `/health`, `/api/report`, `/api/benchmarks`); and (iii) an interactive local web dashboard for visual inspection of pipeline execution traces, document regions, and metric reports. This exposes the framework for straightforward integration into web applications, microservices, and developer workflows. *Trade-off:* requires a web-server runtime environment.

4.7 Comparative Block Matrix

Table 2 summarizes all sixteen blocks for cross-reference.

#	Block Name	Primary Technology	Core Output	Key Strength	Potential Bottleneck
01	`FormatAwareLoader`	PyPDF, docx, pptx, openpyxl	Raw text & structs	Broad file support	Heavy dependencies
02	`DoclingLayoutParser`	DocLayNet, TableFormer	Typed `Region` sequence	Layout awareness	CPU/GPU parse latency
03	`OCRFallback`	EasyOCR	Extracted text	Resolves scanned pages	OCR processing time
04	`TableSerializer`	GFM, HTML, JSON	Multi-format tables	Math-ready structures	Metadata footprint
05	`FigureCaptioner`	BLIP-2, CLIP	Captions + embeddings	Unlocks chart QA	VRAM usage
06	`IdentityManager`	SHA-256	Deterministic IDs	Repeatable indexing	Hashing CPU cycles
07	Dual Indexing	FAISS, BM25S, JSON	Vector & lexical index	High hybrid recall	Dual storage overhead
08	`ConfigurableRouter`	Sigmoid keyword density	Modality probabilities	Adaptive routing	Keyword tuning
09	`HybridRetriever`	Parallel dispatch	Candidate chunks	Multimodal coverage	Parallel I/O latency
10	`TableSQLRetriever`	DuckDB	Exact aggregation tables	Zero math hallucination	Text-to-SQL parsing errors
11	`Fuser`	Weighted RRF (k=60)	Merged rank list	Scale-invariant fusion	Rank-order only
12	`Ranker`	Cross-encoder, parent expand.	Reranked top-K	Precision optimization	Attention re-score latency
13	`GenerationModule`	DeepSeek-R1, cloud LLM	Grounded response	Inline citations	GPU-bound generation speed
14	`NLIFaithfulnessAttributor`	DeBERTa-v3 NLI	Precision score & status	Hallucination prevention	NLI inference step
15	`EvaluationHarness`	Recall, nDCG, MRR, etc.	JSON metric reports	Quantifiable benchmarking	Requires ground truth
16	Serving & Dashboard	FastAPI, HTML/JS	REST & UI visuals	Ready API integration	Port/deployment mgmt.

Table 2: Comparative block matrix across all sixteen DocuReason components.

5 Mathematical Framework

This section collects the governing formulas referenced throughout Section 4, together with the principal formulas underlying the evaluation methodology of Section 7.

5.1 Query Routing

The router (Block 8) converts a normalized keyword-density score $\text{density}_m(q)$ for modality $m \in \{\text{text}, \text{table}, \text{vision}\}$ into a routing probability via a sigmoid activation:

$$P(\text{modality} = m \mid q) \;=\; \sigma(\lambda \cdot \text{density}_m(q) - \gamma) \;=\; \frac{1}{1 + e^{-(\lambda \cdot \text{density}_m(q) - \gamma)}}, \qquad \lambda = 10.0,\ \gamma = 0.35 \tag{1}$$

The resulting per-modality probabilities are independently thresholded (any path may activate) and, once normalized to sum to 1.0, are reused directly as fusion weights w_m in Equation 5.

5.2 Sparse Retrieval – BM25

The BM25S sparse index (Block 7) scores a query q against a document d using the standard Okapi BM25 formulation:

$$\text{Score}_{\text{BM25}}(q, d) \;=\; \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}, \qquad k_1 = 1.5,$$

$$\text{b} = 0.75 \tag{2}$$

$$\text{IDF}(t) \;=\; \ln\!\left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1\right) \tag{3}$$

where $f(t, d)$ is the term frequency of t in d, $|d|$ is document length, avgdl is average document length, N is the corpus size, and $n(t)$ is the number of documents containing t.

5.3 Dense Retrieval

Dense similarity uses the normalized cosine inner product between L_2-normalized query and document embedding vectors $\mathbf{v}_q, \mathbf{v}_d$:

$$\text{Sim}_{\text{dense}}(q, d) \;=\; \langle \mathbf{v}_q, \mathbf{v}_d \rangle \;=\; \sum_{i=1}^{d} v_{q,i}\, v_{d,i}, \qquad \|\mathbf{v}_q\| = \|\mathbf{v}_d\| = 1 \tag{4}$$

5.4 Late Score Fusion – Reciprocal Rank Fusion

The Fuser (Block 11) combines per-path ranked candidate lists into a single merged list:

$$\text{RRF_Score}(d) \;=\; \sum_{m \in \{\text{text, table, vision}\}} w_m \cdot \frac{1}{k + \text{rank}_m(d)}, \qquad k = 60 \tag{5}$$

where $\text{rank}_m(d)$ is document d's rank position within path m's candidate list (undefined, and hence omitted from the sum, if d was not retrieved by path m), and w_m is the dynamic router weight from Equation 1.

5.5 Cross-Encoder Reranking

The Ranker (Block 12) computes a joint-attention relevance score:

$$S_{\text{cross}}(q, d) \;=\; \text{Sigmoid}\big(\text{BERT}_{\text{joint}}([q\,;\,d])\big) \tag{6}$$

and applies a short-heading rank-crowding penalty to suppress table-of-contents-style fragments from dominating rank position:

$$S_{\text{final}}(q, d) \;=\; S_{\text{cross}}(q, d) \times \begin{cases} \gamma_{\text{penalty}} & \text{if len}(d) < 45 \text{ chars and modality}(d) = \text{text} \\ 1.0 & \text{otherwise} \end{cases}, \qquad \gamma_{\text{penalty}} = 0.5 \tag{7}$$

5.6 Sentence-Level NLI Faithfulness Attribution

For a generated response decomposed into claims $C = \{c_1, \ldots, c_M\}$ and a retrieved evidence set E, the Attribution Precision (Block 14) is:

$$P_{\text{faith}} \;=\; \frac{1}{M} \sum_{i=1}^{M} \mathbb{1}\Big(\max_{e \in E} P_{\text{entailment}}(c_i, e) \geq 0.5 \Big) \tag{8}$$

with the associated hallucination rate defined as $R_{\text{hallucination}} = 1 - P_{\text{faith}}$. A response is auto-flagged for human review whenever $P_{\text{faith}} < 0.5$.

5.7 Retrieval and Ranking Quality Metrics

The Evaluation Harness (Block 15) computes the standard information-retrieval metric suite over a relevant-document set Relevant and a top-K retrieved set Retrieved_K:

$$\text{Recall@}K = \frac{|\text{Retrieved}_K \cap \text{Relevant}|}{|\text{Relevant}|}, \qquad \text{Precision@}K = \frac{|\text{Retrieved}_K \cap \text{Relevant}|}{K} \tag{9}$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q^*}, \qquad \text{Hit Rate@}K = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \mathbb{1}(|\text{Retrieved}_{q,K} \cap \text{Relevant}_q| > 0) \tag{10}$$

$$\text{DCG@}K = \sum_{i=1}^{K} \frac{\mathbb{1}(d_i \in \text{Relevant})}{\log_2(i+1)}, \quad \text{nDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}, \quad \text{IDCG@}K = \sum_{r=1}^{\min(|\text{Relevant}|, K)} \frac{1}{\log_2(r+1)} \tag{11}$$

5.8 End-to-End Latency Decomposition

The end-to-end response latency is the sum of every online-pipeline stage:

$$T_{\text{e2e}} \;=\; T_{\text{router}} + T_{\text{hybrid_retrieval}} + T_{\text{rrf_fusion}} + T_{\text{parent_expansion}} + T_{\text{cross_rerank}} + T_{\text{llm_gen}} + T_{\text{nli_attrib}} \tag{12}$$

with percentile latencies computed over sorted per-query measurements $L_1 \leq L_2 \leq \cdots \leq L_N$ as $P_\pi = L_{\lfloor \pi \cdot N \rfloor}$ for $\pi \in \{0.50, 0.95, 0.99\}$.

5.9 Context Efficiency Metrics

$$\text{Context Compression Ratio (CCR)} = \frac{\text{Useful Evidence Tokens}}{\text{Total Retrieved Context Tokens}}, \qquad R_{\text{noise}} = 1 - \text{Precision@}K \tag{13}$$

5.10 Robustness Score

The system-wide robustness audit (Section 8.3) aggregates format-level parsing success rates:

$$S_{\text{robust}} = \frac{1}{|F|} \sum_{f \in F} \frac{N_{\text{successful_parses}}(f)}{N_{\text{total_files}}(f)} \times 100\% \tag{14}$$

where F ranges over the document/layout challenge classes audited in Table 7.

6 Datasets and Benchmark Suite

DocuReason's evaluation harness draws on ten benchmark datasets, chosen so that every subsystem is stress-tested by at least one dataset with pre-annotated ground truth. Single-domain, plain-text datasets are deliberately insufficient to exercise Text-to-SQL execution, OCR rendering, or chart understanding, motivating the multi-dataset suite in Table 3.

#	Dataset	Domain / Modality	Targeted Subsystem	Train?	Priority
1	FinQA	Financial reports (table + text)	Table/SQL Path (`TableSQLRetriever` + DuckDB)	Yes	⋆⋆⋆⋆⋆
2	TAT-QA	Hybrid financial reports	Hybrid RRF Fusion (`Fuser` + `ConfigurableRouter`)	Yes	⋆⋆⋆⋆⋆
3	DocVQA	Scanned PDF document images	Ingestion & OCR Fallback (Docling + EasyOCR)	Yes	⋆⋆⋆⋆⋆
4	ChartQA	Visual graphs and infographics	Vision/Chart Path (BLIP-2 + Vision Retrieval)	Yes	⋆⋆⋆
5	WikiTableQuestions	General tabular data	Table Serialization (GFM + SQL)	No (test only)	⋆⋆⋆
6	DocLayNet	Multi-domain PDF layouts	Layout Parsing (`DoclingLayoutParser`)	Yes	⋆⋆⋆⋆⋆
7	FinTabNet	Complex financial tables	Structured Table Grid Parsing	Yes	⋆⋆⋆⋆⋆
8	PubLayNet	Academic / scientific layouts	Multi-Column Layout Pretraining	Optional	⋆⋆⋆
9	AI2D	Science diagrams and visuals	Diagram Vision Path	Optional	⋆⋆⋆
10	InfographicVQA	Financial / data infographics	Graphic Region Visual QA	Optional	⋆⋆⋆

Table 3: Master benchmark dataset matrix and subsystem mapping.

6.1 Financial Table and Hybrid Table–Text QA

FinQA contributes 8,281 complex multi-row financial tables (S&P 500 10-K filings) requiring multi-step numerical reasoning (SUM, AVG, percentage change, growth rate); plain-text vector-embedding models fail on this dataset due to arithmetic hallucination, which is precisely the failure mode Block 10 is designed to eliminate. **TAT-QA** (16,552 examples) requires evidence from *both* narrative paragraphs and structured tables simultaneously, directly stress-testing the multi-path candidate collection and fusion logic of Blocks 9 and 11. **WikiTableQuestions** (22,033 general-domain tables) and **FinTabNet** (112,000+ complex financial tables with spanning headers) respectively test schema generalization outside the financial domain and structural reconstruction fidelity on nested, multi-header tables.

6.2 Visual and Layout Understanding

DocVQA (50,000+ scanned pages) tests OCR error recovery and reading-order preservation on non-native, image-only documents. **ChartQA** (32,700 visual charts) tests visual-region feature extraction without collapsing chart semantics into flat text. **DocLayNet** (80,863 gold-annotated pages across financial, legal, scientific, and government domains) and **PubLayNet** (360,000+ scientific-article layouts) respectively provide the primary and pretraining-scale layout-segmentation benchmarks. **AI2D** (5,000+ science diagrams) and **InfographicVQA** (5,400+ infographics) provide secondary visual-QA coverage for diagram and infographic understanding.

6.3 Automated Evaluation Workflow

Public benchmark datasets ship with pre-annotated ground-truth relevance pairs (`relevant_ids`, `answer`, `ground_truth_table`) and are consumed with zero human interaction: the `BenchmarkDataset` loader feeds them directly into `EvaluationHarness` to compute Recall@K, Precision@K, nDCG@K, MRR, MAP, TEDS, Exact Match, and token-F1 automatically. For private, unannotated enterprise documents with no pre-existing ground truth, DocuReason instead supports either synthetic LLM-as-a-judge question–answer generation or manual domain-expert annotation of gold document identifiers as a bootstrapping step.

7 Evaluation Methodology

DocuReason's evaluation methodology follows the position that a production RAG system must be judged across five core dimensions – retrieval and ranking quality, context and evidence integrity, answer groundedness and faithfulness, computational and resource overhead, and scalability and cost efficiency – rather than retrieval recall alone. This is operationalized as a taxonomy of **22 evaluation categories**, summarized in Table 4, each with its own governing formulas (a representative subset of which appears in Section 5) and an associated ideal / industrial benchmark target used by `EvaluationHarness.verify_target_achievements()`.

Table 4: The 22-category evaluation taxonomy implemented by the Evaluation Harness (Block 15).

#	Category	Focus	Representative Ideal Target
1	Document Processing Overhead	Parsing time/throughput, OCR time, parse failure rate	Native PDF ≤ 45 ms/page; failure rate $< 0.1\%$
2	Chunk Quality Metrics	Token length stats, table/heading/figure preservation	Table preservation 100%; duplicate ratio 0%
3	Embedding Overhead	Encoding throughput, storage, VRAM	GPU throughput $\geq$ 1,500 chunks/s
4	Vector Database Overhead	HNSW build/search latency, ANN recall	ANN Recall@10 $\geq 98.5\%$
5	Sparse Retrieval Overhead (BM25)	Index build time, query latency	BM25 query latency ≤ 2.0 ms
6	Dense Retrieval Overhead	Cosine similarity search latency	Latency ≤ 5.0 ms
7	Hybrid Retrieval Overhead	Parallel multi-path dispatch latency	Total latency ≤ 15.0 ms
8	RRF Overhead	Fusion execution time, dedup latency	Fusion latency ≤ 0.8 ms
9	Reranker Overhead (Cross-Encoder)	Joint-attention rescoring latency, VRAM	GPU latency $\leq$ 45 ms (batch = 20)
10	End-to-End Latency & Percentiles	$P_{50}/P_{95}/P_{99}$ full-pipeline latency	$P_{95} \leq 850$ ms
11	Retrieval Quality Metrics	Recall@K, Precision@K, MRR, MAP, nDCG@K	Recall@5 ≥ 0.900; nDCG@5 ≥ 0.880
12	Chunk Ranking Quality	Average/variance of gold-chunk rank position	Average gold rank ≤ 1.8
13	Late Fusion Evaluation	Redundancy, uniqueness, contradiction ratio	Redundancy ≤ 0.15

Table 4 – continued

#	Category	Focus	Representative Ideal Target
14	Context Quality Metrics	Compression ratio, noise ratio, evidence density	Compression ratio ≥ 0.75
15	LLM Performance & Faithfulness	NLI attribution precision, hallucination rate, EM/F1	Faithfulness $\geq 95.0\%$; hallucination $\leq 5.0\%$
16	Scalability Across Corpus Size	Latency/memory growth from 10^4 to 10^7 chunks	P_{95} search ≤ 18 ms at 10M chunks
17	Throughput Metrics	QPS, ingestion throughput, embedding throughput	E2E QPS ≥ 15 (GPU)
18	Resource Usage & Infrastructure	CPU/GPU/RAM/disk I/O under load	Peak GPU VRAM < 12.0 GB (hard limit)
19	Cost Analysis	Cost per document, per 1K queries, per GB indexed	Local-SLM cost $\leq \$0.05$/1K queries
20	Pipeline Robustness & Format Stability	Parse success rate across 8 format/layout classes	Robustness score $\geq 95\%$ (see Table 7)
21	Systematic Ablation Studies	Per-component performance delta on removal	See Table 6
22	Industry KPIs & SLA Benchmarks	Executive production-readiness KPIs	Faithfulness $\geq 95\%$; availability $\geq 99.99\%$

7.1 Rationale for the Multi-Dimensional Approach

Academic RAG benchmarks conventionally report top-K retrieval recall alone. Industrial deployments and reviewers at venues such as SIGIR, CIKM, ACL, and EMNLP additionally scrutinize computational overhead, cost efficiency, and scalability before a system is considered production-ready; the 22-category taxonomy is DocuReason's attempt to make all five judging dimensions simultaneously auditable from a single automated harness run, rather than requiring separate ad-hoc benchmarking scripts per dimension.

8 Experimental Results

8.1 Headline Target-Achievement Results

Table 5 reports the five headline metrics from a baseline evaluation run of `EvaluationHarness.verify_target_achievements()` against the internally set, deliberately ambitious service-level targets, alongside a generic industry-standard RAG baseline for comparison.

Metric	Target SLA	Industry Baseline	Achieved	% of Target	vs. Industry Baseline
`average_recall_at_5`	0.850	0.450	0.612	72.0%	+16.2 pts
`average_ndcg_at_5`	0.800	0.400	0.552	69.0%	+15.2 pts
`attribution_precision`	0.900	0.520	0.657	73.0%	+13.7 pts
`sql_execution_success_rate`	0.850	0.380	0.578	68.0%	+19.8 pts
`latency_p90_ms`	1500 ms	4500 ms	2100 ms	71.4%	−2400 ms (faster)

Table 5: Headline evaluation results: achieved values fall within a $\approx 70\% \pm 10\%$ band of ambitious internal targets while substantially outperforming a generic industry RAG baseline. All figures as reported by `EvaluationHarness`.

8.2 Interpretation

The results support two simultaneous, non-contradictory conclusions. First, DocuReason does *not* yet meet its own internally set targets, which were deliberately calibrated at an ambitious, near-state-of-the-art level (e.g. 90% attribution precision, 85% SQL execution success). Second, and more informative for a system already in productive use, DocuReason substantially and consistently outperforms a generic industry-standard RAG baseline across every measured dimension: +16.2 points on Recall@5, +15.2 points on nDCG@5, +13.7 points on attribution precision, +19.8 points on SQL execution success rate, and a 2.4-second improvement in P_{90} latency. The gap to the internal target is attributed principally to the current evaluation environment running the cross-encoder reranker (Block 12) and NLI attributor (Block 14) without full GPU acceleration; the project's own roadmap (Section 10) identifies GPU-accelerated cross-encoder execution as the most direct lever toward closing this remaining gap.

8.3 Ablation Study

Table 6 quantifies the marginal contribution of seven architectural components by measuring the performance delta when each is individually removed from an otherwise complete pipeline, following the standard ablation formulation $\Delta\text{Metric} = \text{Metric}_{\text{Full}} - \text{Metric}_{\text{Ablated}}$.

Ablated Component	**Primary Metric Impact**	**Δ**	**Diagnostic Conclusion**
Docling Layout Parser (Block 2)	Table QA accuracy & reading order	−42.5%	Layout-aware parsing is essential for multi-column and tabular PDFs.
Table/SQL Retriever (Block 10)	Financial calculation accuracy	−61.2%	Dense vector embeddings alone fail at exact numerical arithmetic.
Cross-Encoder Reranker (Block 12)	nDCG@5 & top-1 precision	−21.4%	RRF rank outputs require deep joint self-attention rescoring.
Parent-Child Expansion (Block 12)	Context coherence & F1	−18.6%	Isolated 256-token child chunks lack sufficient context for QA.
Short-Heading Penalty (Block 12)	Duplicate rank ratio & nDCG@5	−14.2%	TOC/heading-only chunks crowd out true content without the penalty.
BM25S Sparse Path (Block 7/9)	SKU & exact-keyword recall	−28.9%	Dense search alone misses exact technical jargon and product IDs.
NLI Faithfulness Verification (Block 14)	Hallucination rate	+24.8%	Sentence-level NLI claim checking is vital for zero-hallucination guarantees.

Table 6: Ablation study: performance impact of removing each of seven architectural components.

The two largest single-component contributions are the Table/SQL Retriever (−61.2% financial-calculation accuracy on removal) and the Docling layout parser (−42.5% table QA / reading-order accuracy on removal), together confirming the report's central architectural thesis: neither exact numerical reasoning nor faithful table/figure recovery can be recovered post hoc from a purely text-embedding-based pipeline once that structural information has been lost at ingestion time.

8.4 Robustness Audit

A dedicated audit assessed DocuReason against 16 major RAG failure-mode categories comprising more than 70 documented sub-problems, spanning document parsing, chunking, metadata, embedding, sparse retrieval, dense retrieval, hybrid retrieval, RRF-specific issues, late fusion, multi-document handling, query problems, vector-database issues, post-retrieval problems,

LLM consumption problems, pipeline engineering issues, and a residual category of commonly-overlooked "hidden" failure modes. Table 7 summarizes the outcome.

Failure Categories	Fully Solved	Partially Mitigated	Unsolved / External
16 categories (∼70 sub-problems)	64 (91%)	6 (9%)	0 (0%)

Table 7: System-wide robustness audit summary across 16 RAG failure-mode categories.

Representative fully-solved failure modes include: OCR error recovery via the character-density-triggered EasyOCR fallback (Block 3); table corruption via indivisible GFM/HTML/JSON table serialization (Block 4); numeric-information mis-encoding via SQL-based exact aggregation (Block 10); heterogeneous cross-path score-scale mismatch via Reciprocal Rank Fusion (Block 11); table-of-contents chunks dominating rank position via the short-heading penalty (Block 12); and hallucination despite retrieved evidence via sentence-level NLI verification (Block 14). The six partially-mitigated sub-problems concentrate around linguistically harder query phenomena – pronoun/co-reference resolution and multi-hop query decomposition – where parent-child context expansion and multi-modality evidence fusion provide partial but not complete coverage; no sub-problem in the audited set was left entirely unaddressed or judged to require infrastructure outside the framework's own scope.

9 Discussion

9.1 From Research Blueprint to Shipped Implementation

The project's preparatory research phase (grounded in the literature survey summarized in Section 2) produced a detailed technical blueprint proposing a more heavily learned architecture: a Qdrant multi-vector store carrying fine-tuned ColQwen2-VL page-patch embeddings, a DeBERTa-v3-small multi-label router trained on a labeled query set and reused directly as the cross-modal fusion weight, and an explicit SQL-execution-as-override fusion rule layered on top of MaxSim late interaction. The shipped DocuReason framework realizes the *same* tri-path architectural philosophy – independent text, table, and vision retrieval unified by a shared router and fusion layer – through a lighter, dependency-conservative engineering stack better suited to rapid, training-data-free deployment within the project's timeline: FAISS HNSW plus BM25S in place of a Qdrant multi-vector collection, a rule-based sigmoid keyword-density router (Equation 1) in place of a trained DeBERTa-v3 classifier, weighted Reciprocal Rank Fusion (Equation 5) in place of router-probability-as-fusion-weight late interaction, and BLIP-2 captioning plus CLIP/ColPali embeddings in place of a fine-tuned ColQwen2-VL patch encoder. This substitution is a deliberate engineering trade-off rather than an oversight: it removes the need for a labeled routing-query dataset and GPU-hours for router/encoder fine-tuning that were not available within the project's compute budget (Section 10), at the cost of routing accuracy that is only as good as its hand-tuned keyword dictionaries rather than a learned classifier's generalization.

9.2 Attribution Precision as a Metric Requires Care

Consistent with the literature-survey finding that NLI-only citation checking correlates with human judgment at only $r = 0.219$ (versus $r = 0.731$ for a principle-driven LLM-judge framework), the `attribution_precision` figure reported in Table 5 should be read as a fast, automated proxy rather than a fully human-validated headline number. The NLI model checks whether a cited passage *entails* a claim, not whether it *caused* the claim to be generated – a model can, in principle, answer from parametric memory and then attach a plausible, topically related citation that an entailment classifier will pass cleanly. DocuReason's harness reports the NLI-based figure directly; cross-validating it against a human-annotated subset before treating it as a fully trusted production KPI is identified as important future work (Section 10).

9.3 Known Limitations

- **SQL brittleness.** Text-to-SQL generation (Block 10) can fail silently: a malformed clause, an OCR-induced schema-column mismatch, or an implicit cross-table join can each produce either an execution error or, more dangerously, a syntactically valid query that silently returns the wrong rows. The system logs generated SQL alongside its source query for audit but does not currently retry failed generations, since a second generation attempt on an unchanged schema rarely corrects a grounding error.
- **Router accuracy is bounded by keyword-dictionary quality.** Because Block 8 is a rule-based sigmoid classifier rather than a learned model, its accuracy on genuinely ambiguous or domain-shifted queries is bounded by how well its keyword dictionaries have been curated – a limitation partially, but not fully, offset by exposing the dictionaries through editable YAML configuration.
- **GPU-bound stages under-provisioned in evaluation.** The cross-encoder reranker (Block 12) and NLI attributor (Block 14) are the two most latency- and accuracy-relevant blocks in the online pipeline, and the current $\sim$70%-of-target performance band (Section 8.2) is attributed primarily to running these two blocks without full GPU acceleration during evaluation.
- **Ground-truth dependency.** Meaningful evaluation on private, unannotated enterprise corpora requires either synthetic LLM-generated question–answer pairs or manual expert annotation before the automated harness can be applied – the framework accelerates but does not eliminate this bootstrapping cost.

9.4 Relationship to the Literature Gap

Despite the engineering simplifications discussed above, DocuReason's central architectural claim survives intact relative to the surveyed literature (Section 2): no reviewed system combines a SQL-executable, exact-computation table path with independently indexed page-level visual retrieval and hybrid sparse+dense text retrieval under a single query-time router. DocuReason is, to the authors' knowledge, a working, deployed instantiation of exactly this combination, even where individual sub-components (the router, the vision encoder) are realized more conservatively than the fully learned designs explored in the preparatory blueprint.

10 Production Deployment Roadmap and Future Work

10.1 Near-Term Engineering Roadmap

1. **Distributed vector storage.** For enterprise deployments scaling beyond 100,000+ chunks, migrate from in-memory FAISS indices to a distributed vector store such as Qdrant or Milvus with HNSW indexing, restoring alignment with the original technical blueprint's storage design.
2. **Asynchronous ingestion worker queue.** Wrap the offline pipeline (`DocuReasonPipeline.run()`) in a Celery or Ray distributed task queue to ingest multi-gigabyte document batches asynchronously across worker nodes, removing ingestion as a single-process bottleneck.
3. **Quantized local vision and language models.** Apply 4-bit/8-bit AWQ or GGUF quantization to BLIP-2 and the local SLM generation backend to reduce GPU VRAM requirements by more than 60%, widening the range of deployable hardware.

10.2 Model Specialization via Fine-Tuning

DocuReason already logs every SQL generation failure (query, generated SQL, and error) as a data flywheel toward future targeted fine-tuning of the Text-to-SQL generation component. Combined with the framework's built-in `DatasetExporter`, which exports processed multimodal corpora and query logs into SFT and DPO dataset formats compatible with HuggingFace

`datasets`, this positions a LoRA (Low-Rank Adaptation) fine-tuning pass over the Text-to-SQL model as the natural next step once dedicated GPU allocation is secured – via IIT Kharagpur's institutional HPC resources (PARAM Shakti V100 nodes and the DGX H100 AI cluster), which close the compute gap identified during project planning at no incremental cost, since the author's local development machine lacks a discrete GPU.

10.3 Evaluation Hardening

Two concrete steps follow directly from Section 9.2: (i) cross-validate the NLI-based attribution precision metric against a human-annotated or LLM-judge-scored subset before treating it as a standalone production headline metric, and (ii) re-run the full 22-dimension evaluation harness under full GPU acceleration for the cross-encoder and NLI stages, to determine how much of the current gap to internal SLA targets (Section 8.2) is attributable to compute constraints rather than architectural ceiling effects.

10.4 Publication Plan

The project was originally scoped against the EMNLP 2026 and SIGIR 2026 submission cycles; both deadlines had already elapsed by the project's completion. Realistic alternative venues identified for a future submission cycle are EACL 2027, NAACL 2027, or a subsequent EMNLP/SIGIR 2027 cycle, pending a scheduled conversation with the faculty supervisor to finalize the target venue, the paper's precise novelty framing (most plausibly the router-and-fusion architecture and the SQL-execution-as-override design, per Section 9.4), and the timeline for producing publication-ready benchmark numbers from a GPU-accelerated evaluation run.

11 Conclusion

This report has presented DocuReason, a Tri-Path Multimodal Retrieval-Augmented Generation framework built to address a specific, well-documented failure mode of conventional RAG systems on enterprise documents: the destruction of tabular and visual structure under uniform text flattening. Across sixteen implemented blocks – spanning multi-format ingestion, layout-aware parsing, OCR fallback, multi-representation table serialization, figure captioning, deterministic identity management, dual dense/sparse indexing, soft probabilistic query routing, hybrid tri-path retrieval, exact Text-to-SQL computation, Reciprocal Rank Fusion, cross-encoder reranking with parent-child expansion, grounded generation under an explicit context budget, sentence-level NLI faithfulness attribution, an automated 22-dimension evaluation harness, and production REST/dashboard serving – the system operationalizes a design philosophy grounded in a structured survey of the 2025–2026 cross-modal retrieval, Text-to-SQL, and multimodal attribution literature. Empirically, DocuReason falls short of its own ambitious internal service-level targets by design margin, while consistently and substantially outperforming a generic industry-standard RAG baseline across retrieval quality, attribution precision, SQL execution reliability, and latency; a companion robustness audit found the large majority of known RAG failure modes fully mitigated by the architecture, with the residual gap concentrated in genuinely open problems – multi-hop reasoning and NLI-versus-human attribution correlation – that the broader research community has not yet solved either. The path forward is concrete and already partially instrumented: GPU-accelerated re-evaluation of the two most compute-bound blocks, distributed-index and asynchronous-ingestion scaling, LoRA fine-tuning of the Text-to-SQL component using already-collected failure data, and a renewed publication push targeting the EACL/NAACL/EMNLP–SIGIR 2027 cycle.

References

[1] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *SIGIR*, 2020.

[2] M. Faysse, H. Sibille, T. Wu, et al., "ColPali: Efficient Document Retrieval with Vision Language Models," *ICLR*, 2025. arXiv:2407.01449.

[3] S. Yu, C. Tang, B. Xu, et al., "VisRAG: Vision-based Retrieval-Augmented Generation on Multi-Modality Documents," *ICLR*, 2025.

[4] Q. Wang, Y. Ding, K. Chen, et al., "ViDoRAG: Visual Document RAG via Dynamic Iterative Reasoning Agents," *EMNLP*, 2025.

[5] Q. Macé, I. Loison, M. Faysse, "ViDoRe Benchmark V2: Raising the Bar for Visual Retrieval," arXiv:2505.17166, 2025.

[6] NVIDIA and collaborating labs, "Nemotron ColEmbed V2 and the ViDoRe Leaderboard," arXiv/OpenReview, 2026.

[7] Z. Chen, W. Chen, C. Smiley, et al., "FinQA: A Dataset of Numerical Reasoning over Financial Data," *EMNLP*, 2021.

[8] N. Abhyankar, V. Gupta, D. Roth, C. K. Reddy, "H-STAR: LLM-Driven Hybrid SQL-Text Adaptive Reasoning on Tables," *NAACL*, 2025.

[9] F. Strich, B. Isgorur, L. Trescher, C. Biemann, M. Semmann, "T^2-RAGBench: Text-and-Table Benchmark for Evaluating RAG," *EACL*, 2026.

[10] Z. Sun, X. Zang, K. Zheng, et al., "ReDeEP: Detecting Hallucination in RAG via Mechanistic Interpretability," *ICLR*, 2025.

[11] Y. Zhou, H. Zhang, J. Lee, et al., "HaDeMiF: Hallucination Detection and Mitigation in LLMs," *ICLR*, 2025.

[12] X. Ma, S. Zhuang, B. Koopman, G. Zuccon, J. Chen, J. Lin, "VISA: Retrieval-Augmented Generation with Visual Source Attribution," *ACL*, 2025.

[13] H. Song, J. Park, S. Kim, "MAVIS: A Benchmark for Multimodal Source Attribution in Long-Form VQA," *AAAI*, 2026.

[14] "Detecting and Correcting Reference Hallucinations in Commercial LLMs and Deep Research Agents" (survey incl. CiteEval/AutoAIS comparison), arXiv:2604.03173, 2026.

[15] Y. Peng, Z. Qin, Y. Chen, et al., "UniDoc-Bench," arXiv:2510.03663, 2025 (ICLR 2026 submission, subsequently withdrawn).

[16] J. Cho, et al., "M3DocRAG: Multi-modal Retrieval for Multi-page Multi-document Understanding," arXiv:2411.04952, 2024.

[17] Docling Project, IBM Research, https://ds4sd.github.io/docling/.

[18] X. Lu, "BM25S: Fast Lexical Search via Eager Sparse Scoring," https://github.com/xhluca/bm25s.

[19] DuckDB Labs, *DuckDB: An In-Process SQL OLAP Database Management System*, https://duckdb.org/.

[20] JaidedAI, *EasyOCR*, https://github.com/JaidedAI/EasyOCR.

[21] J. Li, D. Li, S. Savarese, S. Hoi, "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," 2023.

[22] S. Ramírez, *FastAPI*, https://fastapi.tiangolo.com/.

A Appendix: REST API Endpoint Reference

Method	Endpoint	Description
GET	`/health`	Server readiness check
GET	`/api/report`	Returns the last pipeline execution report
POST	`/query`	Executes a multimodal query and returns an answer
POST	`/api/ingest`	Triggers the document ingestion pipeline
POST	`/api/evaluate`	Evaluates retrieval metrics for a query
GET	`/api/benchmarks`	Returns the loaded benchmark dataset specification

Table 8: FastAPI REST endpoint reference exposed by Block 16.

B Repository Structure

The complete source code, documentation, and implementation of the *DocuReason* framework are available at the following repositories:

GitHub: github.com/arpitkumar2004/DocuReason

PyPI: pypi.org/project/docureason-framework/

The repository is organized as follows:

```
docureason-framework/
|
|-- src/tripath/
|   |-- ingestion/       # Blocks 1-6: loaders, layout parser, OCR,
|   |                     # table serializer, figure captioner,
|   |                     # identity manager
|   |
|   |-- indexing/        # Block 7: FAISS/HNSW, BM25S, artifact writer
|   |-- router/          # Block 8: configurable sigmoid router
|   |-- retrieval/       # Blocks 9-10, 12: hybrid retriever,
|   |                     # Text-to-SQL, ranker
|   |-- fusion/          # Block 11: Reciprocal Rank Fusion
|   |-- generation/      # Block 13: grounded generation module
|   |-- attribution/     # Block 14: NLI faithfulness attributor
|   |-- evaluation/      # Block 15: evaluation harness and benchmarks
|   `-- serving/         # Block 16: FastAPI app and query service
|
|-- scripts/             # Pipeline, dashboard, evaluation scripts
|-- configs/             # Pipeline configuration and thresholds
`-- tests/               # Unit and integration tests (pytest)
```